Policy Gradient Methods

Abstract

This topic introduces policy gradient methods, a class of algorithms used to optimize policies in reinforcement learning. It covers the policy gradient theorem, the implementation of the REINFORCE algorithm, and explores the advantages and challenges of policy gradient methods. By understanding these methods, students will be prepared for more advanced topics such as dynamic programming and deep reinforcement learning.

Learning Objectives

- Understand policy gradient theorem
- Implement the REINFORCE algorithm
- Analyze the advantages and challenges of policy gradient methods
- Understand solutions to problems of policy gradient methods

Motivation & Overview

The Gap: We Need a Learning Method

What we know so far:

- Policies $\pi(a|s)$ map states to actions
- Value functions $V(s)$ and $Q(s, a)$ evaluate policies
- Bellman equations relate values recursively

What we're missing: How do we **learn** an optimal policy?

This Lecture: Policy Gradient Methods

Core idea: $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

What you'll learn:

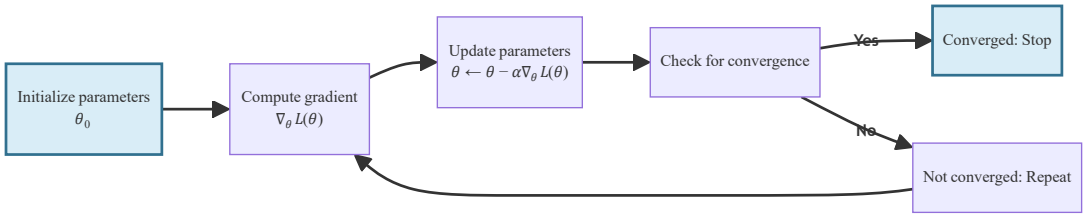
1. Gradient Methods Refresher
2. Policy Gradient Theorem
3. REINFORCE Algorithm
4. Variance Challenge

Why policy gradients?

- Continuous action spaces
- Stochastic policies
- Foundation for modern deep RL

Rehearsal: gradient methods

Overview of gradient descent



Gradient descent principle

Gradient descent is a fundamental algorithm in optimization, where the update rule for parameters θ is given by:

$$\theta \leftarrow \theta - \alpha \nabla_\theta L(\theta)$$

Here, α is the learning rate, and $\nabla_\theta L(\theta)$ is the gradient of the loss function L with respect to the parameters θ.

Loss function minimization

The iterative process of minimizing a loss function $L(\theta)$ can be visualized as:

- **Initialization**: Start with an initial guess for the parameters θ_0.
- **Iteration**: Update the parameters using the rule $\theta_{t+1} = \theta_t - \alpha \nabla_\theta L(\theta_t)$.
- **Convergence**: Continue until the change in $L(\theta)$ is below a predefined threshold.

Optimizer examples

Optimizers like Adam, SGD, and Adagrad enhance the basic gradient descent algorithm by incorporating adaptive learning rates and momentum. These optimizers are crucial in training deep neural networks efficiently:

- **SGD**: Stochastic Gradient Descent, a simple yet effective optimizer.
- **Adagrad**: Adapts the learning rate to the parameters, performing larger updates for infrequent and smaller updates for frequent parameters.
- **RMSProp**: Uses a moving average of squared gradients, preventing the learning rate from becoming too small and maintaining effective training throughout.
- **Adam**: Combines the advantages of two other extensions of stochastic gradient descent, namely AdaGrad and RMSProp.

In [7]:

```
# Helper function for creating animated gradient descent visualiz
ation
# This cell can be hidden/collapsed - it just defines the functio
n

import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML

def create_gradient_descent_animation(theta_history, gradient_his
tory, loss_history, n_iterations):
    """
    Creates an animated visualization showing theta, gradient, an
d loss evolution
    on a single plot with three y-axes.
    """
    # Create figure with single axis
    fig, ax1 = plt.subplots(figsize=(12, 6))
    fig.suptitle('Gradient Descent Animation - Multi-Axis View',
fontsize=16, fontweight='bold')

    # First y-axis: Theta (blue)
    ax1.set_xlabel('Iteration', fontsize=12)
    ax1.set_ylabel(r'$\theta$', color='b', fontsize=12)
    line1, = ax1.plot([], [], 'b-', linewidth=2, label=r'$\theta
(t)$')
    point1, = ax1.plot([], [], 'bo', markersize=8)
    ax1.set_xlim(0, n_iterations)
    ax1.set_ylim(min(theta_history) - 0.5, max(theta_history) +
0.5)
    ax1.tick_params(axis='y', labelcolor='b')
    ax1.grid(True, alpha=0.3)

    # Second y-axis: Gradient (green)
    ax2 = ax1.twinx()
    ax2.set_ylabel(r'$\nabla L(\theta)$', color='g', fontsize=12)
    line2, = ax2.plot([], [], 'g-', linewidth=2, label=r'$\nabla
L(\theta)$')
    point2, = ax2.plot([], [], 'go', markersize=8)
    ax2.axhline(y=0, color='k', linestyle='--', alpha=0.3)
    ax2.set_ylim(min(gradient_history) - 1, max(gradient_history)
+ 1)
    ax2.tick_params(axis='y', labelcolor='g')

    # Third y-axis: Loss (red)
    ax3 = ax1.twinx()
    ax3.spines['right'].set_position(('outward', 60))
    ax3.set_ylabel(r'$L(\theta)$', color='r', fontsize=12)
    line3, = ax3.plot([], [], 'r-', linewidth=2, label=r'$L(\thet
a)$')
    point3, = ax3.plot([], [], 'ro', markersize=8)
    ax3.set_ylim(0, max(loss_history) + 1)
    ax3.tick_params(axis='y', labelcolor='r')

    # Add legend
    lines = [line1, line2, line3]
```

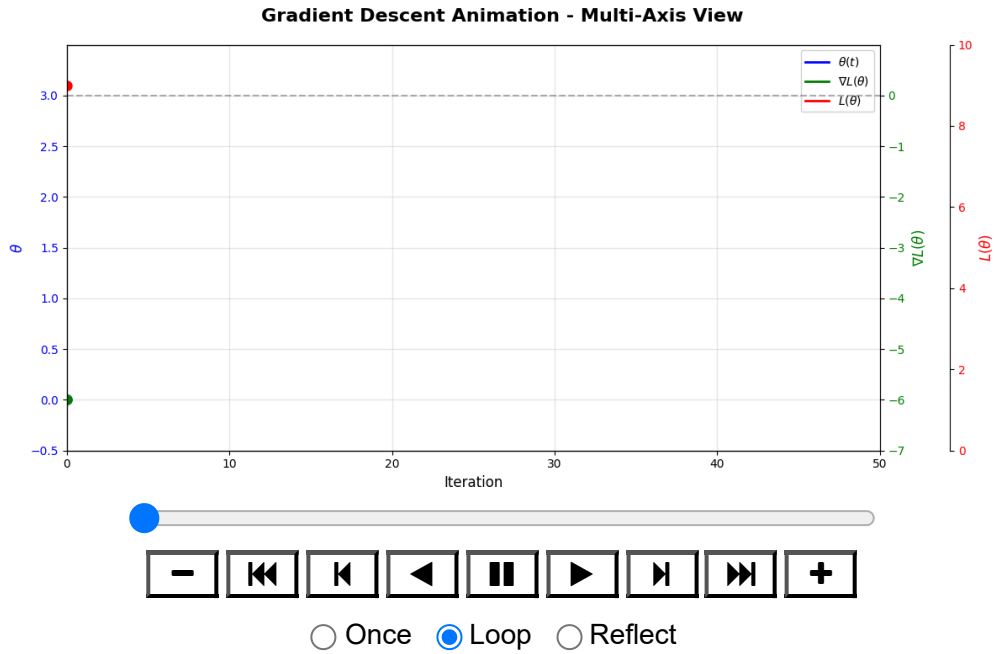
```
    labels = [r'$\theta(t)$', r'$\nabla L(\theta)$', r'$L(\theta)
$']
    ax1.legend(lines, labels, loc='upper right')

    plt.tight_layout()

    # Animation function
    def animate(frame):
        x_data = list(range(frame + 1))

        line1.set_data(x_data, theta_history[:frame+1])
        point1.set_data([frame], [theta_history[frame]])

        line2.set_data(x_data, gradient_history[:frame+1])
        point2.set_data([frame], [gradient_history[frame]])

        line3.set_data(x_data, loss_history[:frame+1])
        point3.set_data([frame], [loss_history[frame]])

        return line1, point1, line2, point2, line3, point3

    # Create animation
    anim = FuncAnimation(fig, animate, frames=n_iterations+1,
                         interval=100, blit=True, repeat=True)

    plt.close(fig)

    return HTML(anim.to_jshtml())
```

In [8]:

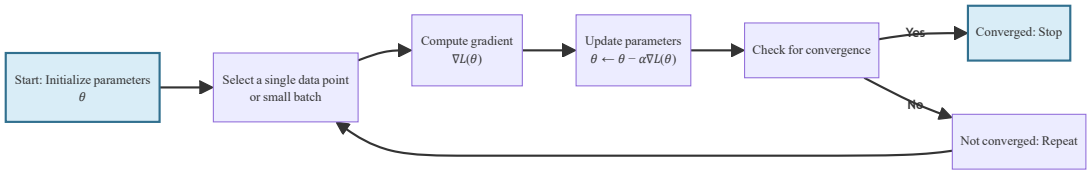
```
# This code implements a simple gradient descent algorithm in Pyt
hon.
# It demonstrates the process of optimizing a quadratic loss func
tion
# by iteratively updating parameters in the direction of the nega
tive gradient.


import numpy as np

# Define the loss function and gradient
def loss_function(theta):
    return (theta - 3) ** 2

def gradient(theta):
    return 2 * (theta - 3)

# Gradient descent parameters
theta_init = 0.0
alpha = 0.1
n_iterations = 50

# Storage for history
theta_history = [theta_init]
gradient_history = [gradient(theta_init)]
loss_history = [loss_function(theta_init)]

# Run gradient descent and store history
theta = theta_init
for i in range(n_iterations):
    grad = gradient(theta)
    theta = theta - alpha * grad

    theta_history.append(theta)
    gradient_history.append(grad)
    loss_history.append(loss_function(theta))

print(f"Recorded {len(theta_history)} iterations")
print(f"Final theta: {theta_history[-1]:.6f}")
print(f"Final loss: {loss_history[-1]:.6f}")

# Create and display the animation
create_gradient_descent_animation(theta_history, gradient_histor
y, loss_history, n_iterations)
```

```
Recorded 51 iterations
Final theta: 2.999957
Final loss: 0.000000
```

Out[8]:



Stochastic gradient descent



SGD leverages the gradient of a single data point or a small batch, which introduces some level of randomness but enhances computational efficiency.

In [9]:
```
def create_animation(x_data, y_data, loss_history, grad_history,
prediction_history, lr_history, target_function, y_lim=(-0.1, 1.
1)):
    """
    Create animation from snapshot-based training history.

    Args:
        x_data, y_data: Lists of arrays (cumulative data per snap
shot)
        loss_history: List of scalars (one per snapshot)
        grad_history: List of scalars (one per snapshot)
        lr_history: List of scalars (one per snapshot)
        prediction_history: List of prediction arrays (one per sn
apshot)
    """
    fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=(1
4, 10))
    fig.suptitle('SGD with Learning Rate Decay', fontsize=16, fon
tweight='bold')

    # Plot 1: Data points and current function
    ax1.set_title('Learned Function vs True Function')
    ax1.set_xlim(0, 2 * np.pi)
    ax1.set_ylim(y_lim[0], y_lim[1])
    ax1.set_xlabel('x')
    ax1.set_ylabel('y')

    x_true = np.linspace(0, 2 * np.pi, 100)
    y_true = target_function(x_true)
    ax1.plot(x_true, y_true, 'k--', alpha=0.3, linewidth=2, label
='True Function')

    line1, = ax1.plot([], [], 'r-', linewidth=2, label='NN Predic
tion')
    scatter = ax1.scatter([], [], c='b', s=30, alpha=0.5, label
='Sampled Data')
    ax1.legend()
    ax1.grid(True, alpha=0.3)

    # Plot 2: Loss over snapshots
    ax2.set_title('Training Loss')
    n_snapshots = len(loss_history)
    ax2.set_xlim(0, n_snapshots)
    max_loss = max(loss_history) if loss_history else 1
    ax2.set_ylim(0, max_loss * 1.1)
    ax2.set_xlabel('Snapshot')
    ax2.set_ylabel('MSE Loss')
    line2, = ax2.plot([], [], 'g-', linewidth=2)
    point2, = ax2.plot([], [], 'go', markersize=8)
    ax2.grid(True, alpha=0.3)

    # Plot 3: Learning Rate Decay
    ax3.set_title('Learning Rate Over Time')
    ax3.set_xlim(0, n_snapshots)
    max_lr = max(lr_history) if lr_history else 1
    ax3.set_ylim(0, max_lr * 1.1)
```

```
    ax3.set_xlabel('Snapshot')
    ax3.set_ylabel('Learning Rate')
    line3, = ax3.plot([], [], 'm-', linewidth=2)
    point3, = ax3.plot([], [], 'mo', markersize=8)
    ax3.grid(True, alpha=0.3)

    # Plot 4: Gradient Magnitude
    ax4.set_title('Gradient Magnitude')
    ax4.set_xlim(0, n_snapshots)
    max_grad = max(grad_history) if grad_history else 1
    ax4.set_ylim(0, max_grad * 1.1)
    ax4.set_xlabel('Snapshot')
    ax4.set_ylabel('Gradient Norm')
    line4, = ax4.plot([], [], 'c-', linewidth=2)
    point4, = ax4.plot([], [], 'co', markersize=8)
    ax4.grid(True, alpha=0.3)

    plt.tight_layout()

    def animate(i):
        # Update learned function
        x_vals = np.linspace(0, 2 * np.pi, 100)
        y_vals = prediction_history[i]
        line1.set_data(x_vals, y_vals)

        # Update data points (cumulative up to this snapshot)
        scatter.set_offsets(np.c_[x_data[i], y_data[i]])

        # Update loss plot (history up to this snapshot)
        snapshot_indices = list(range(i + 1))
        line2.set_data(snapshot_indices, loss_history[:i+1])
        point2.set_data([i], [loss_history[i]])

        # Update learning rate plot
        line3.set_data(snapshot_indices, lr_history[:i+1])
        point3.set_data([i], [lr_history[i]])

        # Update gradient plot
        line4.set_data(snapshot_indices, grad_history[:i+1])
        point4.set_data([i], [grad_history[i]])

        return line1, scatter, line2, point2, line3, point3, line
4, point4

    n_frames = len(prediction_history)
    print(f"Creating animation with {n_frames} frames...")

    anim = FuncAnimation(fig, animate, frames=n_frames, interval=
200, blit=True, repeat=True)
    plt.close(fig)

    print("Converting to HTML...")
    return HTML(anim.to_jshtml())
```

In [10]:

```python
import torch
import torch.nn as nn

class SimpleNN(nn.Module):
    def __init__(self):
        super(SimpleNN, self).__init__()
        self.fc1 = nn.Linear(1, 10)
        self.fc2 = nn.Linear(10, 1)

        nn.init.xavier_uniform_(self.fc1.weight)
        nn.init.xavier_uniform_(self.fc2.weight)
        nn.init.zeros_(self.fc1.bias)
        nn.init.zeros_(self.fc2.bias)

    def forward(self, x):
        x = torch.sigmoid(self.fc1(x))
        x = self.fc2(x)
        x = torch.sigmoid(x)
        return x
```

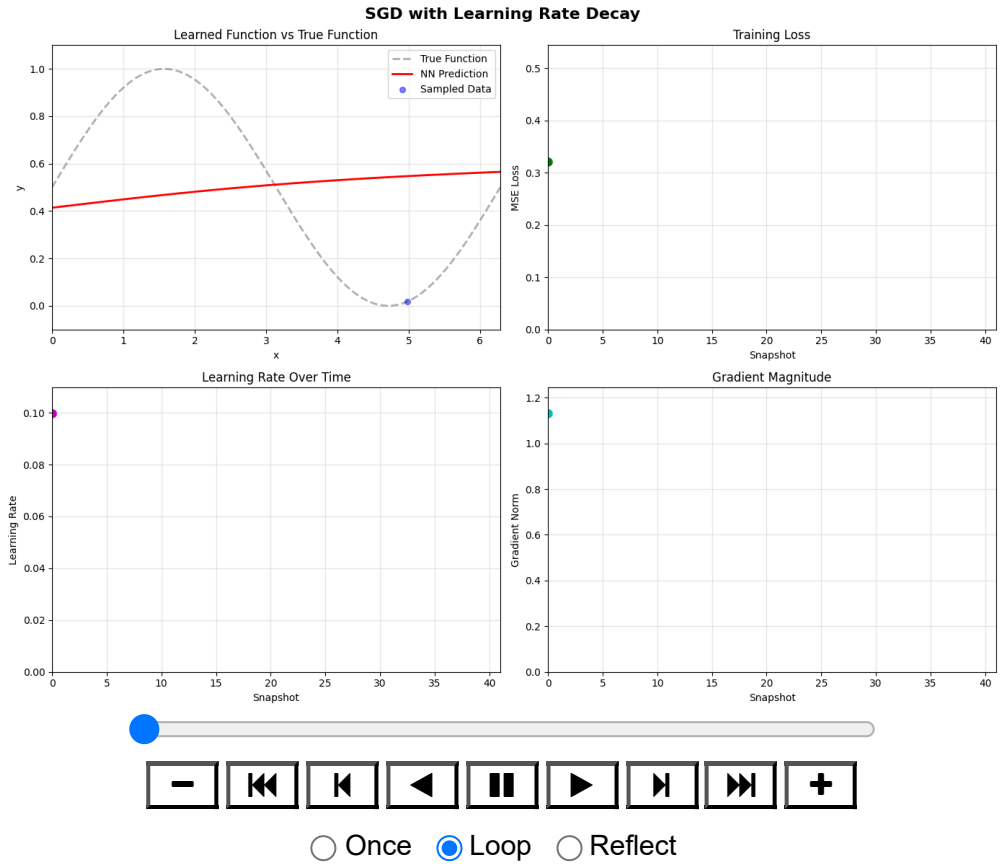
In [5]:

```
"""
Online SGD with simple periodic recording.
Records current state every record_every iterations - no nested h
istory tracking.
"""

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML
from tqdm import tqdm

plt.rcParams['animation.embed_limit'] = 100

# Define the target function - a sigmoid function
def target_function(x):
    """Simple sigmoid function - should be very easy to approxima
te"""
    return np.sin(x)/2 + 0.5
    return 1 / (1 + np.exp(-(x - np.pi)))

def online_sgd(n_iterations=200, record_every=5):
    """
    Run online SGD, recording state every record_every iteration
s.
    One sample per iteration (true online learning).

    Args:
        n_iterations: Total iterations
        record_every: Record state every N iterations
    """
    model = SimpleNN()
    optimizer = optim.SGD(model.parameters(), lr=0.1, momentum=0.
9)
    scheduler = optim.lr_scheduler.ExponentialLR(optimizer, gamma
=0.997)
    criterion = nn.MSELoss()

    # Tracking for all iterations
    x_all, y_all = [], []

    # Recording for animation frames (every record_every iteratio
ns)
    x_snapshots = []
    y_snapshots = []
    loss_snapshots = []
    grad_snapshots = []
    lr_snapshots = []
    prediction_snapshots = []

    for iteration in tqdm(range(n_iterations)):
        # Sample single point (online)
        x_sample = np.random.uniform(0, 2 * np.pi)
```

```
        y_sample = target_function(x_sample)

        x_all.append(x_sample)
        y_all.append(y_sample)

        # Forward and backward
        x_tensor = torch.tensor([[x_sample]], dtype=torch.float3
2)
        y_tensor = torch.tensor([[y_sample]], dtype=torch.float3
2)

        y_pred = model(x_tensor)
        loss = criterion(y_pred, y_tensor)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        scheduler.step()

        # Record snapshot every record_every iterations
        if iteration % record_every == 0 or iteration == n_iterat
ions - 1:
            x_snapshots.append(x_all.copy())
            y_snapshots.append(y_all.copy())
            loss_snapshots.append(loss.item())

            grad_norm = sum(p.grad.norm().item() for p in model.p
arameters() if p.grad is not None)
            grad_snapshots.append(grad_norm)

            lr_snapshots.append(optimizer.param_groups[0]['lr'])

            with torch.no_grad():
                x_vals_tensor = torch.tensor(np.linspace(0, 2 * n
p.pi, 100).reshape(-1, 1), dtype=torch.float32)
                predictions = model(x_vals_tensor).numpy().flatte
n()
                prediction_snapshots.append(predictions)

    print(f"Training: {n_iterations} iterations")
    print(f"Animation frames: {len(prediction_snapshots)} (1 ever
y {record_every} iterations)")

    return x_snapshots, y_snapshots, loss_snapshots, grad_snapsho
ts, prediction_snapshots, lr_snapshots

# Run online SGD
x_data, y_data, loss_history, grad_history, prediction_history, l
r_history = online_sgd(
    n_iterations=200,
    record_every=5
)

create_animation(x_data, y_data, loss_history, grad_history, pred
iction_history, lr_history, target_function)
```

```
100%|██████████| 200/200 [00:00<00:00, 3311.15it/s]

Training: 200 iterations
Animation frames: 41 (1 every 5 iterations)
Creating animation with 41 frames...
Converting to HTML...
```

Out[5]:



Once Loop Reflect

Online and offline GD

Online gradient descent

- Online gradient descent is ideal for environments where data is continuously generated.
- It allows for real-time model updates, which is crucial for applications like stock market prediction and autonomous driving.

Offline gradient descent

- Offline gradient descent is suitable for scenarios where data is static and can be processed in bulk.
- It provides stable convergence but requires significant computational resources.

Mini-batches and replay buffers

> Replay buffers are crucial for breaking the correlation between consecutive experiences, which enhances the stability and performance of reinforcement learning algorithms.

Batch gradient update (mini-batch or full batch):

For a batch of N data points $\{(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)\}$, the gradient update rule becomes:

$$\theta \leftarrow \theta - \alpha \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta L_i(\theta)$$

where:

- $L_i(\theta)$ is the loss computed for the i-th data point
- $\nabla_\theta L_i(\theta)$ is the gradient with respect to that single data point
- The factor $\frac{1}{N}$ averages the gradients across the batch

In [11]:

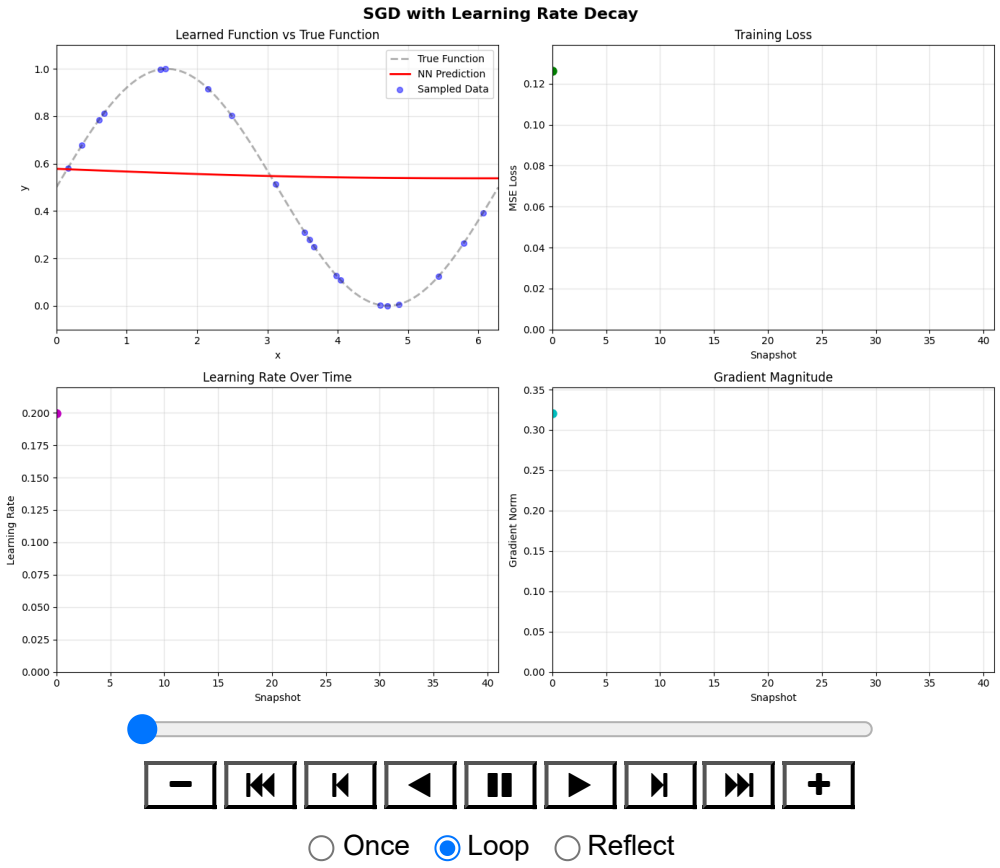
```
"""
Mini-batch SGD with simple periodic recording.
Records current state every record_every iterations - no nested h
istory tracking.
"""

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML
from tqdm import tqdm

plt.rcParams['animation.embed_limit'] = 100

def target_function(x):
    """Simple sigmoid function"""
    return np.sin(x)/2 + 0.5
    return 1 / (1 + np.exp(-(x - np.pi)))


def mini_batch_sgd(dataset_size=100, batch_size=20, n_iterations=
200, record_every=5):
    """
    Run mini-batch SGD, recording state every record_every iterat
ions.

    Args:
        dataset_size: Size of fixed dataset
        batch_size: Samples per mini-batch
        n_iterations: Total iterations
        record_every: Record state every N iterations
    """
    # Generate fixed dataset
    x_dataset = np.random.uniform(0, 2 * np.pi, dataset_size)
    y_dataset = target_function(x_dataset)

    model = SimpleNN()
    optimizer = optim.SGD(model.parameters(), lr=0.2, momentum=0.
9)
    scheduler = optim.lr_scheduler.ExponentialLR(optimizer, gamma
=0.997)
    criterion = nn.MSELoss()

    # Tracking for all iterations
    x_all, y_all = [], []

    # Recording for animation frames (every record_every iteratio
ns)
    x_snapshots = []
    y_snapshots = []
    loss_snapshots = []
    grad_snapshots = []
    lr_snapshots = []
```

```
    prediction_snapshots = []

    for iteration in tqdm(range(n_iterations)):
        # Sample mini-batch
        batch_indices = np.random.choice(dataset_size, batch_siz
e, replace=False)
        x_batch = x_dataset[batch_indices]
        y_batch = y_dataset[batch_indices]

        x_all.extend(x_batch)
        y_all.extend(y_batch)

        # Forward and backward
        x_tensor = torch.tensor(x_batch, dtype=torch.float32).vie
w(-1, 1)
        y_tensor = torch.tensor(y_batch, dtype=torch.float32).vie
w(-1, 1)

        y_pred = model(x_tensor)
        loss = criterion(y_pred, y_tensor)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        scheduler.step()

        # Record snapshot every record_every iterations
        if iteration % record_every == 0 or iteration == n_iterat
ions - 1:
            x_snapshots.append(x_all.copy())
            y_snapshots.append(y_all.copy())
            loss_snapshots.append(loss.item())

            grad_norm = sum(p.grad.norm().item() for p in model.p
arameters() if p.grad is not None)
            grad_snapshots.append(grad_norm)

            lr_snapshots.append(optimizer.param_groups[0]['lr'])
            x_vals_tensor = torch.tensor(np.linspace(0, 2 * np.p
i, 100).reshape(-1, 1), dtype=torch.float32)
            with torch.no_grad():
                predictions = model(x_vals_tensor).numpy().flatte
n()
                prediction_snapshots.append(predictions)

    print(f"Training: {n_iterations} iterations")
    print(f"Animation frames: {len(prediction_snapshots)} (1 ever
y {record_every} iterations)")

    return x_snapshots, y_snapshots, loss_snapshots, grad_snapsho
ts, prediction_snapshots, lr_snapshots

# Run mini-batch SGD
x_data, y_data, loss_history, grad_history, prediction_history, l
r_history = mini_batch_sgd(
    dataset_size=100,
    batch_size=20,
```

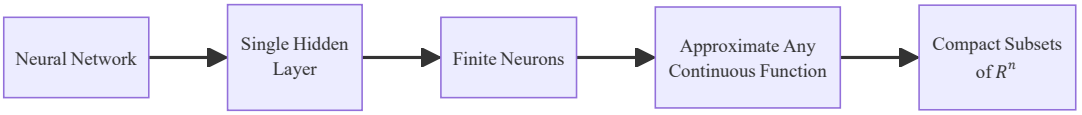
```
    n_iterations=200,
    record_every=5
)

create_animation(x_data, y_data, loss_history, grad_history, pred
iction_history, lr_history, target_function)
```

```
100%|██████████| 200/200 [00:00<00:00, 1387.99it/s]

Training: 200 iterations
Animation frames: 41 (1 every 5 iterations)
Creating animation with 41 frames...
Converting to HTML...
```

Out[11]:



Universal approximation properties



Universal approximation theorem

> The universal approximation theorem highlights the capability of neural networks to serve as universal function approximators. This theorem is fundamental in understanding the power of neural networks in modeling complex functions.

Neural networks as function approximators

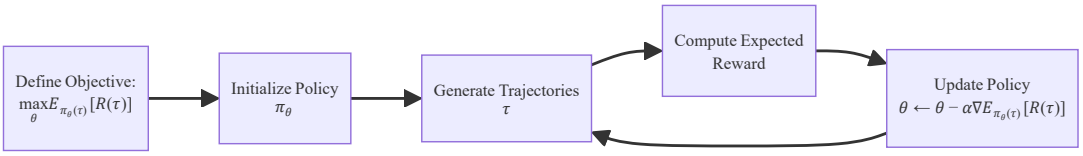
- Neural networks can capture intricate patterns in data, making them suitable for various applications, including reinforcement learning.
- They are particularly useful in scenarios where traditional methods struggle to model complex functions.

Application in RL

Application	Description
Value Function Approximation	Neural networks estimate the expected return of states, aiding in decision-making processes.
Policy Approximation	They help in determining the best actions to take in various states, optimizing the policy.

The policy gradient theorem

Reinforcement Learning as minimization problem



RL as policy optimization

The goal in reinforcement learning is to optimize the policy π_θ to maximize the expected reward:

$$\max_\theta \mathbb{E}_{\pi_\theta(\tau)}[R(\tau)]$$

where θ represents the parameters of the policy, and $R(\tau)$ is the reward obtained from a trajectory τ.

With the gradient method, we have $L(\theta) = \mathbb{E}_{\pi_\theta(\tau)}[R(\tau)]$, so we can update the policy as follows:

$$\theta \leftarrow \theta - \alpha \nabla \mathbb{E}_{\pi_\theta(\tau)}[R(\tau)]$$

[(CS285 Lecture 5: Policy Gradients, Sergey Levine, 2023, p. 2-6) (https://rail.eecs.berkeley.edu/deeprlcourse/deeprlcourse/static/slides/lec-5.pdf)](https://rail.eecs.berkeley.edu/deeprlcourse/deeprlcourse/static/slides/lec-5.pdf)

Evaluating the objective

Objective as expectation

The expectation is taken over all possible trajectories τ, which are sampled according to the policy parameterized by θ, with **discounted rewards**:

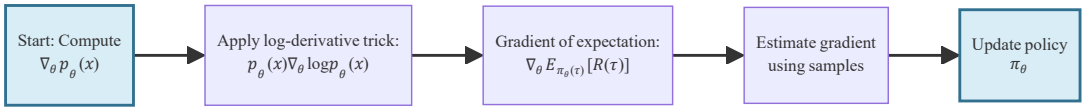
$$J(\theta) = \mathbb{E}_{\pi_\theta(\tau)}\left[\sum_{t=0}^{T} \gamma^t r(s_t, a_t)\right] = \mathbb{E}_{\tau \sim p_\theta(\tau)}\left[\sum_{t=0}^{T} \gamma^t r(s_t, a_t)\right]$$

Key insight: γ is a **hyperparameter** (not learned), doesn't depend on θ. When taking gradients:

$$\nabla_\theta \left[\gamma^t r(s_t, a_t)\right] = \gamma^t \nabla_\theta r(s_t, a_t) = 0$$

Note: The reward doesn't depend on θ either—only the trajectory distribution does.

Direct policy differentiation using the log-derivative trick



Log-derivative identity

The log-derivative trick enables the transformation of gradient expressions:

$$\nabla_\theta p_\theta(x) = p_\theta(x)\nabla_\theta \log p_\theta(x)$$

Gradient of expectation

The gradient of the reinforcement learning objective, when expressed using the log-derivative trick, becomes:

$$J(\theta) = \mathbb{E}_{\tau\sim p_\theta(\tau)}\left[\sum_{t=0}^{T}\gamma^t r(s_t, a_t)\right] = \int_\tau p_\theta(\tau)\left[\sum_{t=0}^{T}\gamma^t r(s_t, a_t)\right] d\tau$$

$$\nabla_\theta J(\theta) = \nabla_\theta \int_\tau p_\theta(\tau)\left[\sum_{t=0}^{T}\gamma^t r(s_t, a_t)\right] d\tau$$

Since γ^t and $r(s_t, a_t)$ don't depend on θ, we can pass ∇_θ through:

$$\nabla_\theta J(\theta) = \int_\tau \nabla_\theta p_\theta(\tau)\left[\sum_{t=0}^{T}\gamma^t r(s_t, a_t)\right] d\tau$$

Applying the **log-derivative trick**:

$$\nabla_\theta J(\theta) = \int_\tau p_\theta(\tau)\nabla_\theta \log p_\theta(\tau)\left[\sum_{t=0}^{T}\gamma^t r(s_t, a_t)\right] d\tau$$

Evaluating the policy gradient (I)

How to estimate the expectation?

$$\mathbb{E}_{\tau \sim \pi_\theta(\tau)}[\dots]$$

Monte Carlo estimation

Monte Carlo estimates the expected discounted return $J(\theta)$ by sampling multiple trajectories and averaging their **discounted** cumulative rewards:

$$J(\theta) = \mathbb{E}_{\pi_\theta(\tau)}\left[\sum_{t=0}^{T} \gamma^t r(s_t, a_t)\right] = \mathbb{E}_{\tau \sim p_\theta(\tau)}\left[\sum_{t=0}^{T} \gamma^t r(s_t, a_t)\right]$$

$$\mathbb{E}_{\tau \sim p_\theta(\tau)}\left[\sum_{t=0}^{T} \gamma^t r(s_t, a_t)\right] \approx \frac{1}{N}\sum_{i=1}^{N}\left(\sum_{t=0}^{T} \gamma^t r(s_{i,t}, a_{i,t})\right)$$

where N is the number of sampled trajectories, and $r(s_t^i, a_t^i)$ represents the reward at time t for the i-th trajectory.

Direct policy differentiation and the policy gradient

Policy gradient theorem (discounted)

The policy gradient theorem states that the gradient of the expected discounted return with respect to the policy parameters θ can be expressed as an expectation over trajectories τ sampled from the policy π_θ. This is given by:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta(\tau)}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot \sum_{t'=0}^{T} \gamma^{t'} r(s_{t'}, a_{t'})\right]$$

with $R(\tau) = \sum_{t=0}^{T} \gamma^t r(s_t, a_t)$ explicitly showing the discounted formulation:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta(\tau)}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot \sum_{t'=0}^{T} \gamma^{t'} r(s_{t'}, a_{t'})\right]$$

$$\nabla_\theta J(\theta) = \int_\tau p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) \left[\sum_{t=0}^{T} \gamma^t r(s_t, a_t) \right] d\tau$$

Consider that $\tau = (s_0, a_0, s_1, a_1, \ldots, s_T, a_T)$. Then:

$$\log p_\theta(\tau) = \log \left[p(s_0) \cdot \prod_{t=0}^{T} \pi(a_t|s_t) \cdot p(s_{t+1}|s_t, a_t) \right]$$

Applying $\log$ rule $\log(a \cdot b) = \log a + \log b$ and considering only terms that depend on θ:

$$\log p_\theta(\tau) = \cancel{\log p(s_0)} + \sum_{t=0}^{T} \log \pi_\theta(a_t|s_t) + \cancel{\log p(s_{t+1}|s_t, a_t)}$$

Substituting $\log p_\theta(\tau) = \sum_{t=0}^{T} \log \pi_\theta(a_t|s_t)$ into $\nabla_\theta J(\theta)$:

$$\nabla_\theta J(\theta) = \int_\tau p_\theta(\tau) \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \left[\sum_{t'=0}^{T} \gamma^{t'} r(s_{t'}, a_{t'}) \right] d\tau$$

Expressing $\int$ as $\mathbb{E}$ again yields the **policy gradient theorem (discounted formulation)**:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot \sum_{t'=0}^{T} \gamma^{t'} r(s_{t'}, a_{t'}) \right]$$

Evaluating the policy gradient

Monte Carlo policy gradient

Substituting the sampling-based average reward in the gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta(\tau)} \left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot \sum_{t'=0}^{T} \gamma^{t'} r(s_{t'}, a_{t'}) \right]$$

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_{i,t}|s_{i,t}) \cdot \sum_{t'=0}^{T} \gamma^{t'} r(s_{i,t'}, a_{i,t'}) \right]$$

Herein, averaging over the $\nabla_\theta \log \pi_\theta(a_{i,t}|s_{i,t})$ is the **likelihood ratio estimation**, since it can be expressed as $\frac{\nabla_\theta \pi_\theta(a_{i,t}|s_{i,t})}{\pi_\theta(a_{i,t}|s_{i,t})}$ (the ratio of the gradient to the probability).

Reward to go and causality

Reward-to-go and causality

Causality in policy gradient methods ensures that only **future rewards** are considered when computing the gradient at a given time step t. This principle is essential to avoid biasing the gradient estimate with rewards that are not influenced by the current action.

With discounting, the causality constraint uses the **reward-to-go** G_t:

$$G_t = \sum_{k=t}^{T} \gamma^{k-t} \cdot r_{k+1}$$

The policy gradient with reward-to-go becomes:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta(\tau)} \left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot \sum_{t'=t}^{T} \gamma^{t'} r(s_{t'}, a_{t'}) \right]$$

Note: $\sum_{t'=t}^{T} \gamma^{t'} r(s_{t'}, a_{t'}) = \gamma^t G_t$, where the discount factor γ^t accounts for how far into the future these rewards occur.

Monte Carlo approximation:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_{i,t}|s_{i,t}) \cdot \sum_{t'=t}^{T} \gamma^{t'} r(s_{i,t'}, a_{i,t'}) \right]$$

This is exactly what the **REINFORCE** algorithm implements! Each action's gradient is weighted by the discounted sum of future rewards, ensuring policy updates reflect the actual long-term consequences of actions.

The REINFORCE algorithm

Overview of the REINFORCE algorithm

REINFORCE definition

Algorithm: REINFORCE

Input:

- Differentiable policy $\pi_\theta(a|s)$ parameterized by θ
- Learning rate α
- Discount factor γ

Initialize policy parameters θ arbitrarily

For each episode:

1. Generate an episode (trajectory) following π_θ:
   $\tau = \{s_0, a_0, r_1, s_1, a_1, r_2, \ldots, s_{T-1}, a_{T-1}, r_T\}$
2. For each time step t in the episode (t = 0, 1, ..., T-1):

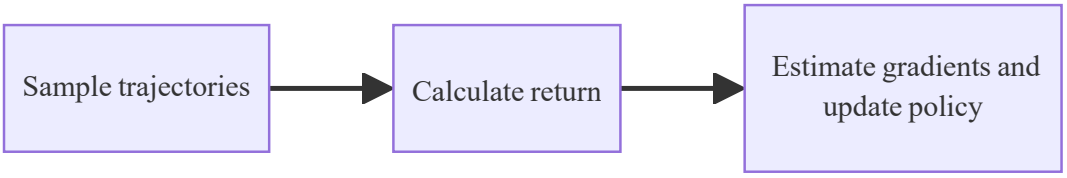
   a. Calculate the return (reward-to-go) from time t: $G_t = \sum_{k=t}^{T} \gamma^{k-t} \cdot r_{k+1}$

   b. Update policy parameters using gradient ascent: $\theta \leftarrow \theta + \alpha \cdot \gamma^t \cdot G_t \cdot \nabla_\theta \log \pi_\theta(a_t|s_t)$

Return optimized policy π_θ

Key Components:

- Trajectory sampling: Generate complete episodes using the current policy
- Return calculation: Compute discounted cumulative rewards from each time step
- Policy gradient: Update parameters in the direction that increases expected return
- Monte Carlo: Uses complete episode returns (no bootstrapping)



In [7]:

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import gymnasium as gym
from collections import deque
import time

# Hidden cell: Test policy visualization function
def test_policy(policy_net, env_name='CartPole-v1', max_steps=50
0, n_test_episodes=3, render=True):
    """
    Test the trained policy by running one episode with rendering
enabled.

    Parameters:
    - policy_net: Trained PolicyNetwork instance
    - env_name: Name of the environment to test on
    - max_steps: Maximum number of steps per episode

    Returns:
    - total_reward: Total reward accumulated during the episode
    """
    import logging

    logger = logging.getLogger(__name__)
    logger.setLevel(logging.DEBUG)
    if render:
        env = gym.make(env_name, render_mode="human")
    else:
        env = gym.make(env_name, render_mode="rgb_array")
    avg_test_reward = 0
    for n_ep in range(n_test_episodes):
        state, _ = env.reset()
        total_reward = 0
        done = False
        steps = 0

        logger.info(f"Testing policy on {env_name}")

        while not done and steps < max_steps:
            # Convert state to tensor
            state_tensor = torch.FloatTensor(state).unsqueeze(0)

            # Get action probabilities from policy
            with torch.no_grad():
                action_probs = policy_net(state_tensor)

            # Select action with highest probability (greedy)
            action = torch.argmax(action_probs).item()

            # Take action in environment
            next_state, reward, terminated, truncated, _ = env.st
ep(action)
            done = terminated or truncated
```

```
            total_reward += reward
            state = next_state
            steps += 1

        logger.info(f"Episode finished after {steps} steps with t
otal reward: {total_reward}")
        avg_test_reward += total_reward

    env.close()

    avg_test_reward /= n_test_episodes

    return avg_test_reward

# Analyze and visualize gradient variance from REINFORCE training
import numpy as np
import matplotlib.pyplot as plt

def compute_gradient_statistics(grad_history, max_params_to_show=
20):
    """
    Compute variance and other statistics for gradients across ep
isodes.

    Parameters:
    - grad_history: List of gradient arrays for each episode
    - max_params_to_show: Maximum number of parameter elements to
display

    Returns:
    - Dictionary with gradient statistics
    """
    num_episodes = len(grad_history)

    # Flatten all gradients into single vectors per episode
    flat_grads = []
    for ep in range(num_episodes):
        episode_grads = np.concatenate([g for g in grad_history[e
p]])
        flat_grads.append(episode_grads)

    flat_grads = np.array(flat_grads)
    total_params = flat_grads.shape[1]

    # Compute statistics across episodes for each parameter eleme
nt
    param_means = np.mean(flat_grads, axis=0)
    param_variances = np.var(flat_grads, axis=0)

    # Limit to max_params_to_show if specified
    if max_params_to_show and max_params_to_show < total_params:
        top_indices = np.argsort(param_variances)[-max_params_to_
show:][::-1]
        param_means = param_means[top_indices]
        param_variances = param_variances[top_indices]
        display_label = f"Top {max_params_to_show} (by variance)"
    else:
```

```
        top_indices = None
        display_label = "All parameters"

    return {
        'means': param_means,
        'variances': param_variances,
        'total_params': total_params,
        'displayed_params': len(param_variances),
        'num_episodes': num_episodes,
        'display_label': display_label,
        'top_indices': top_indices,
        'flat_grads': flat_grads
    }

def plot_gradient_variance(grad_history, max_params_to_show=20):
    """
    Create visualizations of gradient variance across training.

    Parameters:
    - grad_history: List of gradient arrays for each episode
    - max_params_to_show: Maximum number of individual parameters
to show in bar plots
    """
    stats = compute_gradient_statistics(grad_history, max_params_
to_show)

    fig, axes = plt.subplots(2, 2, figsize=(14, 10))
    fig.suptitle(f'REINFORCE Gradient Variance Analysis\n({stats
["display_label"]} of {stats["total_params"]} total)',
                 fontsize=16, fontweight='bold')

    # Plot 1: Variance per parameter element
    ax1 = axes[0, 0]
    ax1.bar(range(stats['displayed_params']), stats['variances'],
color='steelblue', alpha=0.7)
    ax1.set_xlabel('Parameter Element Index')
    ax1.set_ylabel('Gradient Variance')
    ax1.set_title(f'Variance by Parameter Element ({stats["displa
y_label"]})')
    ax1.grid(True, alpha=0.3)

    # Plot 2: Gradient magnitude over episodes
    ax2 = axes[0, 1]
    grad_norms = [np.linalg.norm(np.concatenate([p for p in grad_
history[ep]]))
                  for ep in range(len(grad_history))]
    ax2.plot(grad_norms, linewidth=1.5, color='darkgreen', alpha=
0.7)
    ax2.set_xlabel('Episode')
    ax2.set_ylabel('Gradient L2 Norm (all parameters)')
    ax2.set_title('Total Gradient Magnitude Over Training')
    ax2.grid(True, alpha=0.3)

    # Plot 3: Rolling variance of gradient norms
    ax3 = axes[1, 0]
    window_size = 50
    if len(grad_norms) >= window_size:
```

```
        rolling_var = [np.var(grad_norms[max(0, i-window_size):i+
1])
                       for i in range(len(grad_norms))]
        ax3.plot(rolling_var, linewidth=1.5, color='purple', alph
a=0.7)
        ax3.set_xlabel('Episode')
        ax3.set_ylabel(f'Rolling Variance (window={window_siz
e})')
        ax3.set_title('Gradient Magnitude Variance Over Time')
        ax3.grid(True, alpha=0.3)
    else:
        ax3.text(0.5, 0.5, f'Need at least {window_size} episodes
\nfor rolling variance',
                 ha='center', va='center', transform=ax3.transAxe
s)

    # Plot 4: Distribution of gradient magnitudes (histogram)
    ax4 = axes[1, 1]
    ax4.hist(grad_norms, bins=30, color='coral', alpha=0.7, edgec
olor='black')
    ax4.axvline(np.mean(grad_norms), color='red', linestyle='--',
linewidth=2,
                label=f'Mean: {np.mean(grad_norms):.3f}')
    ax4.axvline(np.median(grad_norms), color='blue', linestyle='-
-', linewidth=2,
                label=f'Median: {np.median(grad_norms):.3f}')
    ax4.set_xlabel('Gradient Magnitude')
    ax4.set_ylabel('Frequency')
    ax4.set_title('Distribution of Gradient Magnitudes')
    ax4.legend()
    ax4.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

# Define the Policy Network π_ϑ(a|s)
class PolicyNetwork(nn.Module):
    """
    Differentiable policy π_ϑ(a|s) parameterized by ϑ.
    Neural network with one hidden layer.
    """
    def __init__(self, state_dim, action_dim, hidden_dim=128):
        super(PolicyNetwork, self).__init__()
        self.fc1 = nn.Linear(state_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, action_dim)

    def forward(self, state):
        """Forward pass to compute action probabilities."""
        x = torch.relu(self.fc1(state))
        action_logits = self.fc2(x)
        return torch.softmax(action_logits, dim=-1)
```

In [8]:
```
# REINFORCE Algorithm Implementation for CartPole

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import gymnasium as gym
from collections import deque

# REINFORCE Algorithm
def reinforce(env_name='CartPole-v1', num_episodes=300, learning_
rate=0.01, gamma=0.99, hidden_dim=128, eval_every_n = 100, max_st
eps=300, render=False):
    """
    REINFORCE Algorithm Implementation

    Input:
      - env_name: Environment to train on
      - learning_rate (α): Learning rate for gradient ascent
      - gamma (γ): Discount factor
      - num_episodes: Number of training episodes
      - hidden_dim: Size of hidden layer in policy network
    """

    # Initialize environment
    env = gym.make(env_name)
    state_dim = env.observation_space.shape[0]
    action_dim = env.action_space.n

    # Initialize policy parameters ϑ arbitrarily
    policy_net = PolicyNetwork(state_dim, action_dim, hidden_dim)
    optimizer = optim.Adam(policy_net.parameters(), lr=learning_r
ate)

    # Track episode rewards for monitoring
    episode_rewards = deque(maxlen=100)
    grad_history = [] # To store gradients for visualization
    # For each episode:
    for episode in range(num_episodes):

        # 1. Generate an episode (trajectory) following π_ϑ
        state, _ = env.reset()
        trajectory = []  # τ = {s_0, a_0, r_1, s_1, a_1, r_2,
...}
        log_probs = []   # Store log π_ϑ(a_t|s_t) for gradient co
mputation
        rewards = []     # Store rewards r_{t+1}
        done = False
        steps = 0
        while not done and steps < max_steps:
            # Convert state to tensor
            state_tensor = torch.FloatTensor(state).unsqueeze(0)

            # Get action probabilities from policy π_ϑ(a|s)
            action_probs = policy_net(state_tensor)
```

```
            # Sample action from the distribution
            dist = torch.distributions.Categorical(action_probs)
            action = dist.sample()

            # Store log π_ϑ(a_t|s_t) for gradient computation
            log_probs.append(dist.log_prob(action))

            # Take action in environment
            next_state, reward, terminated, truncated, _ = env.st
ep(action.item())
            done = terminated or truncated

            # Store reward
            rewards.append(reward)

            # Move to next state
            state = next_state

            # Increase step counter
            steps += 1

        # 2. For each time step t in the episode:

        # 2a. Calculate the return (reward-to-go) from time t:
        # G_t = Σ_{k=t}^{T} γ^{k-t} * r_{k+1}
        returns = []
        G = 0
        for r in reversed(rewards):
            G = r + gamma * G
            returns.insert(0, G)

        returns = torch.FloatTensor(returns)

        # 2b. Update policy parameters using gradient ascent:
        # ϑ ← ϑ + α * γ^t * G_t * ∇_ϑ log π_ϑ(a_t|s_t)
        policy_loss = []
        for t, (log_prob, G_t) in enumerate(zip(log_probs, return
s)):
            # Note: We multiply by -1 because optimizers minimiz
e, but we want to maximize
            # The γ^t term is often omitted in practice as it's a
constant factor
            policy_loss.append(-log_prob * G_t)

        # Perform gradient ascent (via backpropagation)
        optimizer.zero_grad()
        # Summation over time steps
        policy_loss = torch.stack(policy_loss).sum()
        policy_loss.backward()
        # Record gradients for each parameter after backward pass
        grads = [p.grad.detach().cpu().numpy().copy().flatten() f
or p in policy_net.parameters()]
        grad_history.append(grads)
        optimizer.step()

        # Track progress
        episode_reward = sum(rewards)
```

```
        episode_rewards.append(episode_reward)

        # Print progress every n episodes
        if (episode + 1) % eval_every_n == 0:
            avg_reward = np.mean(episode_rewards)
            print(f"Episode {episode + 1}/{num_episodes}, Average
Reward (last {eval_every_n}): {avg_reward:.2f}")
            test_reward = test_policy(policy_net, render=render,
n_test_episodes=2, max_steps=max_steps)
            print(f"Average test episode reward: {test_reward}")

    env.close()

    # Return optimized policy π_ϑ
    return policy_net, grad_history
```

In [9]:

```python
# Train the REINFORCE agent
print("Training REINFORCE agent with neural network policy on Car
tPole-v1...")
print("=" * 70)
trained_policy, grad_history = reinforce(
    env_name='CartPole-v1',
    num_episodes=1000,
    learning_rate=0.002,
    gamma=0.99,
    hidden_dim=16,
    max_steps=300,
    render=True
)
print("=" * 70)
print("Training complete!")
# Print total number of elements in gradient history (sum of arra
y sizes)
total_grad_elements = sum(arr.size for episode_grads in grad_hist
ory for arr in episode_grads)
print(f"Gradient history size (number of elements): {total_grad_e
lements}")
```

```
Training REINFORCE agent with neural network policy on CartPole-v
1...
=================================================================
=====
Episode 100/1000, Average Reward (last 100): 28.12
Average test episode reward: 64.5
Episode 200/1000, Average Reward (last 100): 51.68
Average test episode reward: 300.0
Episode 300/1000, Average Reward (last 100): 96.44
Average test episode reward: 300.0
Episode 400/1000, Average Reward (last 100): 214.50
Average test episode reward: 300.0
Episode 500/1000, Average Reward (last 100): 211.26
Average test episode reward: 300.0
Episode 600/1000, Average Reward (last 100): 241.48
Average test episode reward: 246.0
Episode 700/1000, Average Reward (last 100): 243.06
Average test episode reward: 278.0
Episode 800/1000, Average Reward (last 100): 289.23
Average test episode reward: 300.0
Episode 900/1000, Average Reward (last 100): 280.19
Average test episode reward: 300.0
Episode 1000/1000, Average Reward (last 100): 292.58
Average test episode reward: 300.0
=================================================================
=====
Training complete!
Gradient history size (number of elements): 114000
```

Strengths of the REINFORCE algorithm

Simplicity and generality

- REINFORCE's simplicity allows for straightforward implementation in various reinforcement learning scenarios.
- Its generality ensures compatibility with any differentiable, parameterized stochastic policy, enhancing its applicability across different problems.

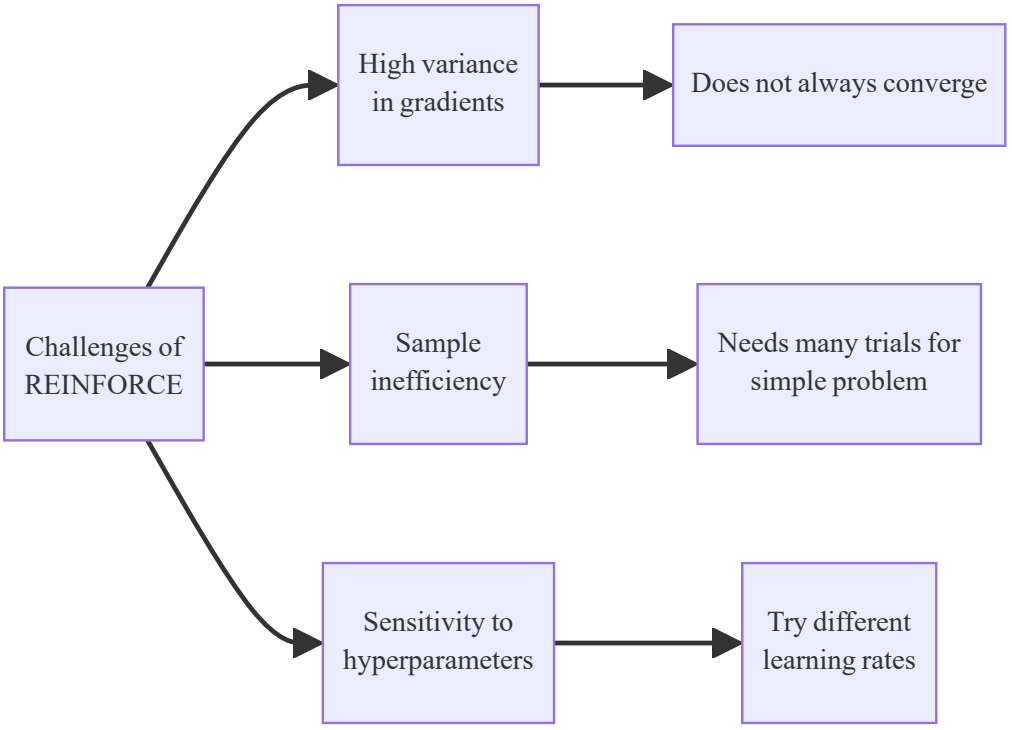
Unbiased gradient estimates

> The unbiased nature of REINFORCE's gradient estimates guarantees that, on average, the policy updates lead to performance improvements, making it a reliable choice for policy optimization.

Applicability to various problems

Feature	Benefit
Discrete and Continuous Action Spaces	Flexibility in handling a wide range of reinforcement learning tasks.
No Dynamics Knowledge Required	Enables application in environments where the dynamics are unknown or complex.

Limitations and challenges



High variance in gradients

> The high variance in gradient estimates is a significant challenge in the REINFORCE algorithm, often resulting in slow convergence and instability during training. This issue necessitates the use of variance reduction techniques to improve learning efficiency.

Inefficiency in sample usage

- On-policy nature: REINFORCE requires new samples for each update, leading to high sample complexity.
- Sample inefficiency: This limitation makes it less practical for environments where data collection is expensive.

Sensitivity to hyperparameters

Hyperparameter	Impact
Learning Rate	Too high can cause divergence; too low can slow down learning.
Batch Size	Influences the stability and speed of convergence.

Testing a different learning rate

In [10]:
```
# Train the REINFORCE agent
print("Training REINFORCE agent with neural network policy on Car
tPole-v1...")
print("=" * 70)
trained_policy, grad_history_bad_lr = reinforce(
    env_name='CartPole-v1',
    num_episodes=1000,
    learning_rate=0.02, # !!!CHANGED FROM 0.002 TO 0.02!!!
    gamma=0.99,
    hidden_dim=16,
    max_steps=300,
    render=False
)
print("=" * 70)
print("Training complete!")
# Print total number of elements in gradient history (sum of arra
y sizes)
total_grad_elements = sum(arr.size for episode_grads in grad_hist
ory for arr in episode_grads)
print(f"Gradient history size (number of elements): {total_grad_e
lements}")
```

```
Training REINFORCE agent with neural network policy on CartPole-v
1...
=================================================================
=====
Episode 100/1000, Average Reward (last 100): 49.39
Average test episode reward: 251.5
Episode 200/1000, Average Reward (last 100): 107.42
Average test episode reward: 49.0
Episode 300/1000, Average Reward (last 100): 170.92
Average test episode reward: 300.0
Episode 400/1000, Average Reward (last 100): 277.93
Average test episode reward: 300.0
Episode 500/1000, Average Reward (last 100): 159.75
Average test episode reward: 115.5
Episode 600/1000, Average Reward (last 100): 252.72
Average test episode reward: 300.0
Episode 700/1000, Average Reward (last 100): 274.31
Average test episode reward: 221.0
Episode 800/1000, Average Reward (last 100): 297.05
Average test episode reward: 300.0
Episode 900/1000, Average Reward (last 100): 300.00
Average test episode reward: 300.0
Episode 1000/1000, Average Reward (last 100): 300.00
Average test episode reward: 300.0
=================================================================
=====
Training complete!
Gradient history size (number of elements): 114000
```

Investigating gradient variance for the appropriate learning rate

In [11]:

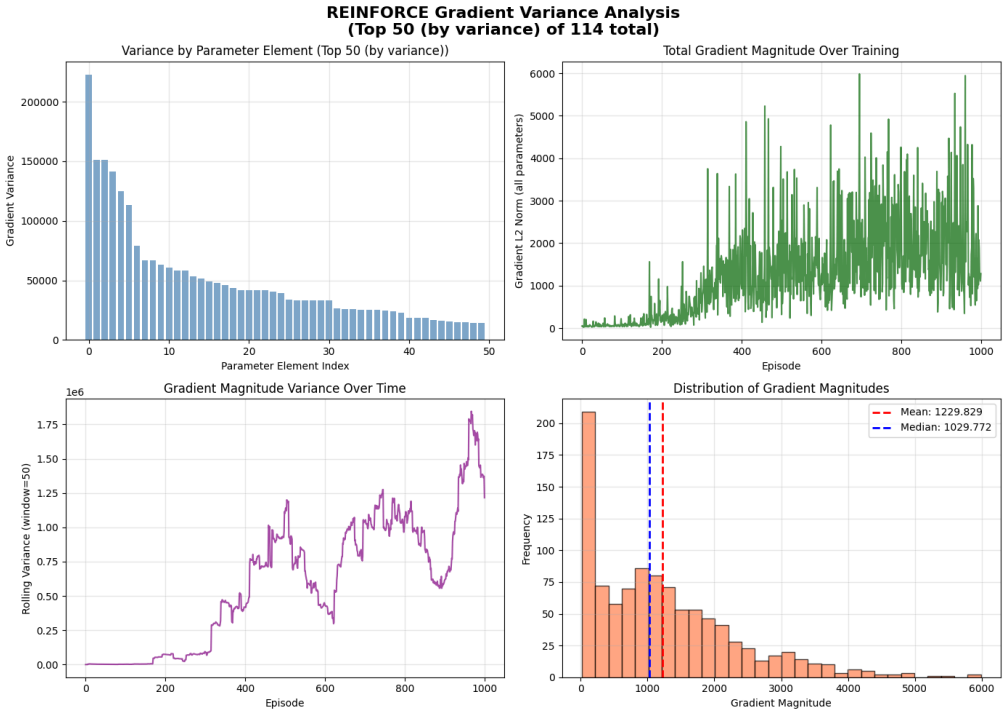
```
# Analyze and visualize gradient variance from REINFORCE training
import numpy as np
import matplotlib.pyplot as plt

def compute_gradient_statistics(grad_history, max_params_to_show=
20):
    """
    Compute variance and other statistics for gradients across ep
isodes.

    Parameters:
    - grad_history: List of gradient arrays for each episode
    - max_params_to_show: Maximum number of parameter elements to
display

    Returns:
    - Dictionary with gradient statistics
    """
    num_episodes = len(grad_history)

    # Flatten all gradients into single vectors per episode
    flat_grads = []
    for ep in range(num_episodes):
        episode_grads = np.concatenate([g for g in grad_history[e
p]])
        flat_grads.append(episode_grads)

    flat_grads = np.array(flat_grads)
    total_params = flat_grads.shape[1]

    # Compute statistics across episodes for each parameter eleme
nt
    param_means = np.mean(flat_grads, axis=0)
    param_variances = np.var(flat_grads, axis=0)

    # Limit to max_params_to_show if specified
    if max_params_to_show and max_params_to_show < total_params:
        top_indices = np.argsort(param_variances)[-max_params_to_
show:][::-1]
        param_means = param_means[top_indices]
        param_variances = param_variances[top_indices]
        display_label = f"Top {max_params_to_show} (by variance)"
    else:
        top_indices = None
        display_label = "All parameters"

    return {
        'means': param_means,
        'variances': param_variances,
        'total_params': total_params,
        'displayed_params': len(param_variances),
        'num_episodes': num_episodes,
        'display_label': display_label,
        'top_indices': top_indices,
        'flat_grads': flat_grads
    }
```

```
def plot_gradient_variance(grad_history, max_params_to_show=20):
    """
    Create visualizations of gradient variance across training.

    Parameters:
    - grad_history: List of gradient arrays for each episode
    - max_params_to_show: Maximum number of individual parameters
to show in bar plots
    """
    stats = compute_gradient_statistics(grad_history, max_params_
to_show)

    fig, axes = plt.subplots(2, 2, figsize=(14, 10))
    fig.suptitle(f'REINFORCE Gradient Variance Analysis\n({stats
["display_label"]} of {stats["total_params"]} total)',
                 fontsize=16, fontweight='bold')

    # Plot 1: Variance per parameter element
    ax1 = axes[0, 0]
    ax1.bar(range(stats['displayed_params']), stats['variances'],
color='steelblue', alpha=0.7)
    ax1.set_xlabel('Parameter Element Index')
    ax1.set_ylabel('Gradient Variance')
    ax1.set_title(f'Variance by Parameter Element ({stats["displa
y_label"]})')
    ax1.grid(True, alpha=0.3)

    # Plot 2: Gradient magnitude over episodes
    ax2 = axes[0, 1]
    grad_norms = [np.linalg.norm(np.concatenate([p for p in grad_
history[ep]]))
                  for ep in range(len(grad_history))]
    ax2.plot(grad_norms, linewidth=1.5, color='darkgreen', alpha=
0.7)
    ax2.set_xlabel('Episode')
    ax2.set_ylabel('Gradient L2 Norm (all parameters)')
    ax2.set_title('Total Gradient Magnitude Over Training')
    ax2.grid(True, alpha=0.3)

    # Plot 3: Rolling variance of gradient norms
    ax3 = axes[1, 0]
    window_size = 50
    if len(grad_norms) >= window_size:
        rolling_var = [np.var(grad_norms[max(0, i-window_size):i+
1])
                       for i in range(len(grad_norms))]
        ax3.plot(rolling_var, linewidth=1.5, color='purple', alph
a=0.7)
        ax3.set_xlabel('Episode')
        ax3.set_ylabel(f'Rolling Variance (window={window_siz
e})')
        ax3.set_title('Gradient Magnitude Variance Over Time')
        ax3.grid(True, alpha=0.3)
    else:
        ax3.text(0.5, 0.5, f'Need at least {window_size} episodes
\nfor rolling variance',
```

```
                ha='center', va='center', transform=ax3.transAxe
s)

    # Plot 4: Distribution of gradient magnitudes (histogram)
    ax4 = axes[1, 1]
    ax4.hist(grad_norms, bins=30, color='coral', alpha=0.7, edgec
olor='black')
    ax4.axvline(np.mean(grad_norms), color='red', linestyle='--',
linewidth=2, label=f'Mean: {np.mean(grad_norms):.3f}')
    ax4.axvline(np.median(grad_norms), color='blue', linestyle='-
-', linewidth=2, label=f'Median: {np.median(grad_norms):.3f}')
    ax4.set_xlabel('Gradient Magnitude')
    ax4.set_ylabel('Frequency')
    ax4.set_title('Distribution of Gradient Magnitudes')
    ax4.legend()
    ax4.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()
```

In [12]:

```
# Visualize gradient variance from REINFORCE training
# Use max_params_to_show=None to show all parameters
plot_gradient_variance(grad_history, max_params_to_show=50)
```



The variance problem

Problem: The variance of gradients over time is quite high, causing instabilities in learning.

Why does REINFORCE suffer from high variance?

Reward-to-go helps (summing only future rewards from each time step), but three fundamental sources of variance remain:

Gradient Variance Source 1: Monte Carlo Sampling Variance

The fundamental problem: We use a **finite sample of episodes** to estimate expectations over **all possible trajectories**.

What we need:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta(\tau)} \left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot \sum_{t'=t}^{T} r(s_{t'}, a_{t'}) \right]$$

What we compute (basic REINFORCE, $N = 1$):

$$\nabla_\theta J(\theta) \approx \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot \sum_{t'=t}^{T} r(s_{t'}, a_{t'})$$

Why this creates variance: The same policy π_θ produces drastically different trajectories due to:

1. **Stochastic actions:** Policy samples different actions from its probability distribution
2. **Stochastic environment:** Random transitions lead to different outcomes

CartPole example - same policy, different outcomes:

Same policy, pole tilted left at step 50:

- **Episode A:** Samples "move left" (70% prob), stabilizes, T=300, return = 250
- **Episode B:** Samples "move right" (30% prob), falls, T=150, return = 100
- **Episode C:** Same action as A, but physics noise (0.01° vs 0.00°), T=180, return = 130

Identical policy yields returns from 100 to 250 - gradient "points in different directions" across episodes.

Solution: Mini-Batch Averaging

Average gradients over N episodes before updating:

Gradient Variance Source 2: Credit Assignment Uncertainty

The problem: When computing the gradient at time t, we weight the log-probability by the **entire reward-to-go**:

$$\nabla_\theta \log \pi_\theta(a_t|s_t) \cdot \sum_{t'=t}^{T} r(s_{t'}, a_{t'})$$

This assigns equal credit to action a_t for **all future rewards**, even rewards primarily caused by later actions.

Why this creates variance:

Consider two actions in the same episode:

- Action at $t = 10$: $\sum_{t'=10}^{300} r(s_{t'}, a_{t'}) = 290$ (episode continues 290 more steps)
- Action at $t = 11$: $\sum_{t'=11}^{300} r(s_{t'}, a_{t'}) = 289$ (same episode, one step later)

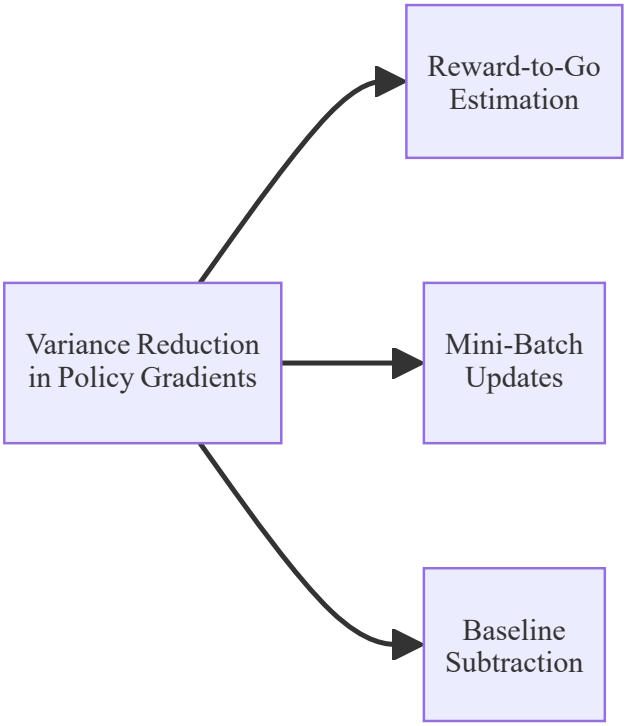
Both actions receive nearly identical return values, even though:

- The action at $t = 10$ might be critical (keeps pole balanced)
- The action at $t = 11$ might be bad or mediocre (just happens to be in a good episode)

The algorithm has no way to assign the credits to the good action and to penalize the bad action!

Solutions: Baseline Subtraction. Instead of asking "how much total reward did we get?", ask "how much **more** reward than expected did we get?"

Variance reduction techniques



Reward-to-go estimator

The reward-to-go estimator focuses on future rewards, effectively reducing variance in policy gradient estimates while maintaining the unbiased nature of the gradient.

Mini-batch gradient updates

The variance of the gradient estimate decreases with batch size:

$$\mathrm{Var}\left[\frac{1}{N}\sum_{i=1}^{N}\nabla_i\right] = \frac{1}{N}\cdot\mathrm{Var}[\nabla_{\text{single episode}}]$$

where N is the number of episodes in the mini-batch. Larger batches provide more stable gradient estimates at the cost of requiring more samples per update.

Baseline subtraction

Baseline subtraction modifies the policy gradient to:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}\nabla_\theta \log \pi_\theta(a_t|s_t)(G_t - b_t)\right]$$

where b_t is the baseline. Common choices:

Baseline Type	Description	Usage
Normalization	Mean return across per episode per std. dev.	Simple, widely used

Baseline Type	Description	Usage
Average reward	Mean return across episodes	Simple, widely used
Value function	Learned estimate $V(s_t)$	Actor-critic methods
Optimal baseline	Minimizes variance theoretically	Rarely used (complex)

In [13]:
```
def reinforce_minibatch_normalization(
    env_name='CartPole-v1',
    num_episodes=1000,
    batch_size=10,
    learning_rate=0.002,
    gamma=0.99,
    hidden_dim=16,
    eval_every_n=100,
    normalize_returns=False,
    max_steps=300,
    render=False
):
    """
    Mini-batch REINFORCE with loss averaging over rolling window.

    Collects one episode at a time, maintains a rolling buffer of
the last
    batch_size episode losses, averages them, and updates the pol
icy.
    Same number of episodes as batch_size=1, but with variance re
duction.
    """
    env = gym.make(env_name)
    state_dim = env.observation_space.shape[0]
    action_dim = env.action_space.n

    policy_net = PolicyNetwork(state_dim, action_dim, hidden_dim)
    optimizer = optim.Adam(policy_net.parameters(), lr=learning_r
ate)

    episode_rewards = deque(maxlen=100)
    grad_history = []
    batch_losses = deque(maxlen=batch_size)

    for episode in range(num_episodes):
        state, _ = env.reset()
        log_probs = []
        rewards = []
        done = False
        steps = 0
        while not done and steps < max_steps:
            state_tensor = torch.FloatTensor(state).unsqueeze(0)
            action_probs = policy_net(state_tensor)
            dist = torch.distributions.Categorical(action_probs)
            action = dist.sample()

            log_probs.append(dist.log_prob(action))

            next_state, reward, terminated, truncated, _ = env.st
ep(action.item())
            done = terminated or truncated
            rewards.append(reward)
            state = next_state
            steps += 1

        episode_rewards.append(sum(rewards))
```

```
        # Calculate returns for this episode
        returns = []
        G = 0
        for r in reversed(rewards):
            G = r + gamma * G
            returns.insert(0, G)
        returns = torch.FloatTensor(returns)

        # Normalize returns
        if normalize_returns:
            returns = (returns - returns.mean()) # / (returns.std
() + 1e-8)

        # Compute loss for this episode
        episode_loss = []
        for log_prob, G_t in zip(log_probs, returns):
            episode_loss.append(-log_prob * G_t)

        # Sum over timesteps and detach from computation graph be
fore storing
        episode_loss_scalar = torch.stack(episode_loss).sum().det
ach()
        batch_losses.append(episode_loss_scalar)

        # Recompute current episode loss with computation graph f
or backprop
        current_loss = torch.stack(episode_loss).sum()

        # Average current loss with detached historical losses
        if len(batch_losses) > 1:
            historical_avg = torch.stack(list(batch_losses)[:-
1]).mean()
            avg_loss = (current_loss + historical_avg * (len(batc
h_losses) - 1)) / len(batch_losses)
        else:
            avg_loss = current_loss

        # Backpropagate and update
        optimizer.zero_grad()
        avg_loss.backward()

        # Record gradients
        grads = [p.grad.detach().cpu().numpy().copy().flatten()
                for p in policy_net.parameters() if p.grad is not
None]
        grad_history.append(grads)

        optimizer.step()

        if (episode + 1) % eval_every_n == 0:
            avg_reward = np.mean(episode_rewards)
            print(f"Episode {episode + 1}/{num_episodes}, Avg Rew
ard: {avg_reward:.2f}")
            test_reward = test_policy(policy_net, render=render,
max_steps=max_steps)
            print(f"Average test episode reward: {test_reward:.2
```

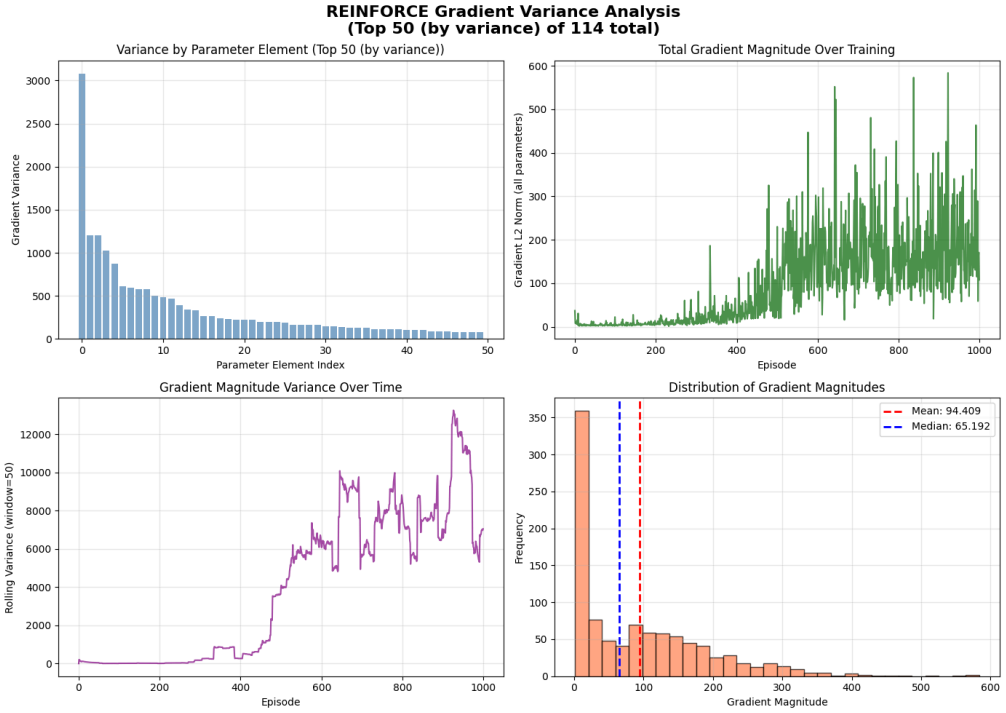
```
f}")

    env.close()
    return policy_net, grad_history

print("Training REINFORCE with batch_size")
print("=" * 70)
```

```
Training REINFORCE with batch_size
======================================================================
=====
```

In [14]:

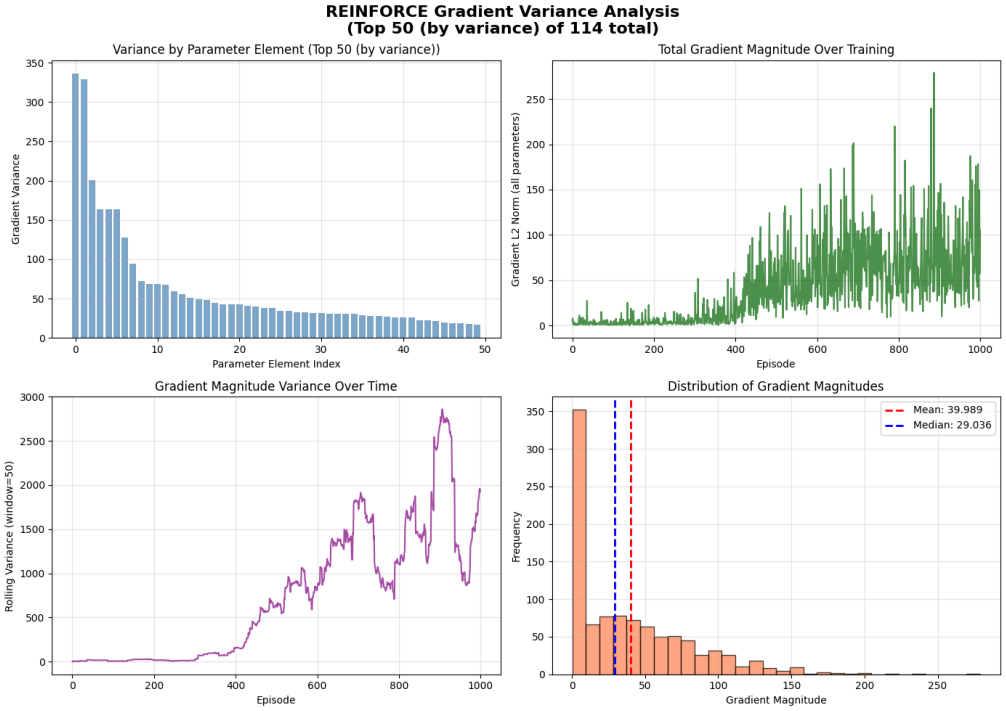
```
print("Training REINFORCE with mini batches...")
print("=" * 70)

trained_policy_batch, grad_history_batch = reinforce_minibatch_no
rmalization(
    env_name='CartPole-v1',
    num_episodes=1000,
    batch_size=10,
    learning_rate=0.002,
    gamma=0.99,
    hidden_dim=16,
    eval_every_n=100,
    normalize_returns=False,
    max_steps=300,
    render=False
)

print("=" * 70)
print("Training complete")

total_grad_elements = sum(arr.size for episode_grads in grad_hist
ory_batch for arr in episode_grads)
print(f"Gradient history size: {total_grad_elements}")

# Visualize gradient variance from REINFORCE training
# Use max_params_to_show=None to show all parameters
plot_gradient_variance(grad_history_batch, max_params_to_show=50)
```

```
Training REINFORCE with mini batches...
================================================================
=====
Episode 100/1000, Avg Reward: 23.36
Average test episode reward: 24.33
Episode 200/1000, Avg Reward: 29.51
Average test episode reward: 73.33
Episode 300/1000, Avg Reward: 42.05
Average test episode reward: 163.67
Episode 400/1000, Avg Reward: 65.00
Average test episode reward: 197.33
Episode 500/1000, Avg Reward: 117.01
Average test episode reward: 300.00
Episode 600/1000, Avg Reward: 233.14
Average test episode reward: 300.00
Episode 700/1000, Avg Reward: 262.22
Average test episode reward: 300.00
Episode 800/1000, Avg Reward: 267.86
Average test episode reward: 300.00
Episode 900/1000, Avg Reward: 281.87
Average test episode reward: 300.00
Episode 1000/1000, Avg Reward: 290.11
Average test episode reward: 300.00
================================================================
=====
Training complete
Gradient history size: 114000
```



In [16]:
```
print("Training REINFORCE with batch size and normalization...")
print("=" * 70)

trained_policy_baseline, grad_history_batch_normalization = reinf
orce_minibatch_normalization(
    env_name='CartPole-v1',
    num_episodes=1000,
    batch_size=10,
    learning_rate=0.002,
    gamma=0.99,
    hidden_dim=16,
    eval_every_n=100,
    normalize_returns=True,
    max_steps=300,
    render=False
)

print("=" * 70)
print("Training complete")

plot_gradient_variance(grad_history_batch_normalization, max_para
ms_to_show=50)
```

```
Training REINFORCE with batch size and normalization...
================================================================
=====
Episode 100/1000, Avg Reward: 26.19
Average test episode reward: 40.00
Episode 200/1000, Avg Reward: 33.11
Average test episode reward: 69.00
Episode 300/1000, Avg Reward: 37.97
Average test episode reward: 84.33
Episode 400/1000, Avg Reward: 58.04
Average test episode reward: 162.00
Episode 500/1000, Avg Reward: 118.94
Average test episode reward: 300.00
Episode 600/1000, Avg Reward: 157.40
Average test episode reward: 300.00
Episode 700/1000, Avg Reward: 199.72
Average test episode reward: 300.00
Episode 800/1000, Avg Reward: 272.51
Average test episode reward: 300.00
Episode 900/1000, Avg Reward: 290.77
Average test episode reward: 300.00
Episode 1000/1000, Avg Reward: 296.88
Average test episode reward: 300.00
================================================================
=====
Training complete
```



Summary: Policy Gradient Methods

Core Mathematical Framework

Policy Gradient Objective:

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\sum_t r(s_t, a_t) \right]$$

Policy Gradient Theorem:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta(\tau)} \left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot R(\tau) \right]$$

Monte Carlo Approximation:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_{i,t}|s_{i,t}) \cdot \sum_{t'=t}^{T} r(s_{i,t'}, a_{i,t'}) \right]$$

REINFORCE Algorithm

Update Rule:

$$\theta \leftarrow \theta + \alpha \cdot G_t \cdot \nabla_\theta \log \pi_\theta(a_t|s_t)$$

where $G_t = \sum_{k=t}^{T} \gamma^{k-t} \cdot r_{k+1}$ is the discounted return from time t.

Strengths:

- Simple and general - works with any differentiable policy
- Unbiased gradient estimates
- Applicable to discrete and continuous action spaces
- Model-free (no environment dynamics required)

Limitations:

- **High variance** in gradient estimates
- **Sample inefficient** (on-policy, cannot reuse data)
- **Sensitive to hyperparameters** (learning rate, batch size)

Two Sources of Gradient Variance

1. **Monte Carlo Sampling Variance**: Finite episodes estimate infinite trajectory distribution
    - Same policy produces vastly different returns due to stochasticity
    - Example: CartPole returns range 100-250 from identical policy
2. **Credit Assignment Uncertainty**: All future rewards weighted equally

- Action at $t = 10$ gets credit for all 290 future rewards
- Cannot distinguish critical actions from mediocre ones in good episodes

Variance Reduction Techniques

Technique		Formula	Effect
Reward-to-Go	$\sum_{t'=t}^{T} r(s_{t'}, a_{t'})$	instead of $\sum_{t'=0}^{T} r(s_{t'}, a_{t'})$	Removes past rewards (unbiased)
Mini-Batch Averaging	$\mathrm{Var}[\frac{1}{N}\sum \nabla_i] = \frac{1}{N}\mathrm{Var}[\nabla]$		Reduces variance by $1/N$
Baseline Subtraction	$(G_t - b_t)$	instead of G_t	Unbiased variance reduction
Return Normalization (special case of baseline subtraction)		$\frac{G_t - \mu}{\sigma}$	Stabilizes gradient magnitudes

Experimental Observations

From CartPole-v1 experiments:

- Learning rate 0.002 converges; 0.02 diverges (10x sensitivity) for vanilla REINFORCE
- Gradient variance decreases dramatically with mini-batches (batch_size=10)
- Gradient variance decreases further with return normalization
- With batch learning and return normalization, robustness to hyperparameter variations is significantly improved (now learning rate 0.02 converges)

Summary

This notebook provides a comprehensive introduction to **policy gradient methods** in reinforcement learning, with a focus on the **REINFORCE algorithm**. The key topics covered include:

1. Gradient Methods Foundation

- Review of gradient descent principles and optimization techniques
- Stochastic gradient descent (SGD) and mini-batch learning
- Neural networks as universal function approximators
- Online vs. offline learning with replay buffers

2. Policy Gradient Theorem

- Formulation of RL as an optimization problem: $\max_\theta \mathbb{E}_{\pi_\theta(\tau)}[R(\tau)]$
- Derivation using the **log-derivative trick**: $\nabla_\theta p_\theta(x) = p_\theta(x)\nabla_\theta \log p_\theta(x)$
- Policy gradient expression: $\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta(\tau)}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot R(\tau)\right]$
- Monte Carlo estimation for practical implementation

3. REINFORCE Algorithm

- Step-by-step implementation of the REINFORCE algorithm
- Hands-on CartPole-v1 environment experiments
- Reward-to-go and causality considerations
- Strengths: simplicity, generality, unbiased estimates
- Limitations: high variance, sample inefficiency, hyperparameter sensitivity

4. Variance Problem Analysis

- **Source 1**: Monte Carlo sampling variance from finite episodes
- **Source 2**: Credit assignment uncertainty across time steps
- Detailed gradient variance visualizations from training
- Impact of hyperparameters (learning rate sensitivity)

5. Variance Reduction Solutions

- **Mini-batch averaging**: Variance reduction by $1/N$
- **Return normalization**: Stabilizing gradient magnitudes
- **Baseline subtraction**: Unbiased variance reduction (general principle)
- Experimental validation showing improved stability and convergence

Key Takeaways

- Policy gradient methods directly optimize parameterized policies
- REINFORCE provides unbiased but high-variance gradient estimates

- Variance reduction techniques are essential for practical performance
- Mini-batching and normalization significantly improve robustness to hyperparameters
- These foundations prepare for advanced methods like Actor-Critic and PPO

What's Next: Value-Based Methods and Beyond

This notebook introduced **policy gradient methods**, which directly optimize the policy $\pi_\theta(a|s)$. However, this is just one approach to solving reinforcement learning problems.

Two Main Paradigms in Reinforcement Learning

1. Policy-Based Methods (This Lecture)

- Directly optimize the policy without explicitly learning value functions
- REINFORCE algorithm covered in this notebook
- Better for continuous action spaces and stochastic policies
- Suffers from high variance

2. Value-Based Methods (Lectures 4-6)

These methods first learn value functions, then derive policies from them:

- **Lecture 4: Dynamic Programming (DP)**
  - Policy Evaluation and Policy Improvement
  - Value Iteration and Policy Iteration
  - Requires complete knowledge of environment dynamics
  - Computationally intensive but guaranteed convergence
- **Lecture 5: Monte Carlo (MC) Methods**
  - Model-free learning from complete episodes
  - Estimates value functions through sampling
  - No bootstrapping - waits for actual returns
- **Lecture 6: Temporal Difference (TD) Learning**
  - Combines ideas from DP (bootstrapping) and MC (sampling)
  - SARSA (on-policy) and Q-Learning (off-policy)
  - Updates estimates using other estimates (bootstrapping)
  - More sample-efficient than MC, more flexible than DP

Bridging the Gap: Actor-Critic Methods (Lectures 7+)

Actor-Critic algorithms **merge both approaches**:

Key Advantages:

- **Actor** (policy network): Handles continuous action spaces naturally
- **Critic** (value network): Reduces variance by providing baseline
- Combines strengths of both paradigms while mitigating weaknesses